

MOLEKUL: A Pure-Python Ab Initio Quantum Chemistry Platform for Education and Reproducible Benchmarking

Enis Yazici*

*SRH University of Applied Sciences Heidelberg,
School of Engineering and Architecture, Campus Heidelberg,
Ludwig-Guttmann-Straße 6, 69123 Heidelberg, Germany*

Abstract

MOLEKUL is an open-source, pure-Python implementation of Restricted Hartree–Fock (RHF) self-consistent field theory and second-order Møller–Plesset perturbation theory (MP2) for closed-shell molecules. Starting from a molecular geometry and a contracted Gaussian basis set, MOLEKUL evaluates all one- and two-electron integrals analytically, solves the Roothaan–Hall equations iteratively, and reports the total RHF and MP2 energies. Built-in basis data cover hydrogen through fluorine for STO-3G, 6-31G*, and cc-pVDZ. Additional modules provide a geometry optimizer and Mulliken and Löwdin population analysis. Validated against PySCF on a 14-molecule benchmark suite, MOLEKUL achieves RHF/STO-3G total-energy agreement within $5 \times 10^{-8} E_h$ and MP2 correlation-energy agreement within $2 \times 10^{-7} E_h$. The codebase comprises 606 automated tests and is designed for advanced undergraduate and graduate courses in quantum chemistry, as well as for reproducible algorithmic inspection and validation.

Keywords: quantum chemistry, Hartree–Fock, MP2, ab initio, Gaussian basis sets, Python, education

Table 1: Code Metadata

Nr.	Detail
C1	Current code version: v0.1.2
C2	Repository: https://github.com/yazicienis/molekul
C3	Reproducible capsule (Zenodo): https://doi.org/10.5281/zenodo.19763107
C4	License: MIT
C5	Versioning system: git
C6	Languages and tools: Python ≥ 3.10 , NumPy
C7	Dependencies: numpy; pytest (development)
C8	Documentation: https://github.com/yazicienis/molekul
C9	Support: enis.yazici@srh.de

*Corresponding author

Email address: enis.yazici@srh.de (Enis Yazici)

1. Motivation and Significance

Production electronic-structure packages such as PySCF [1], Psi4 [2], and ORCA [3] provide broad functionality and optimized implementations, but their internal complexity can obscure the connection between textbook equations and executable algorithms. Earlier educational codes such as PyQuante [4] demonstrated the value of readable implementations, but they are no longer as actively maintained or systematically validated against modern reference packages.

MOLEKUL occupies the space between textbook equations and production code. Every algorithmic step traces directly to a named Python function with clear correspondence to standard quantum-chemistry references [5, 6]. The 606-test suite and deterministic NumPy implementation ensure that every numerical claim is automatically reproducible.

Beyond pedagogy, the Python/NumPy implementation provides a self-contained baseline for performance profiling, because quantum-chemistry algorithms are implemented explicitly rather than delegated to compiled electronic-structure kernels, allowing clean isolation of NumPy/BLAS performance characteristics.

2. Software Description

2.1. Architecture

MOLEKUL is organized as an installable Python package (`src/molekul/`). Table 2 lists the main modules. Installation: `pip install -e ".[dev]"`. Tests: `pytest tests/`.

Table 2: Main modules of MOLEKUL.

Module	Responsibility
<code>integrals.py</code>	One-electron integrals
<code>eri.py</code>	Two-electron repulsion integrals
<code>rhf.py</code>	RHF SCF driver
<code>mp2.py</code>	MP2 correlation energy
<code>population.py</code>	Mulliken and Löwdin analysis
<code>optimizer.py</code>	Geometry optimizer
<code>harmonic.py</code>	Harmonic frequencies (experimental)

2.2. Integral Engine

One- and two-electron integrals are evaluated analytically using the McMurchie–Davidson scheme [7, 6], reducing all integral evaluation to a Boys-function call $F_n(x) = \int_0^1 t^{2n} e^{-xt^2} dt$ [8].

The Boys function is evaluated using a pure-Python combination of `math.erf` for the analytic F_0 expression, recurrence relations, and small- x Taylor expansions, with the branch selected to maintain numerical stability. A dedicated test confirms relative errors below 5×10^{-14} across the argument range encountered in STO-3G through cc-pVDZ bases; no SciPy dependency is required.

ERIs are stored in a full N_{AO}^4 double-precision array—dense storage is intentional, keeping the Fock-build loop transparent. At $N_{\text{AO}} = 100$ this requires approximately 800 MB; practical use is limited to small molecules. Contracted shells are explicitly renormalized to unit overlap, correcting for cross-primitive contributions commonly omitted in introductory treatments.

2.3. SCF Driver

The RHF SCF driver [9, 10] implements: (i) a Superposition of Atomic Densities (SAD) initial guess with spherically-averaged neutral-atom occupations; (ii) DIIS acceleration [11, 12] using the error vector $\mathbf{e} = \mathbf{FPS} - \mathbf{SPF}$; and (iii) an optional level shift [13] that stabilizes early iterations. Default thresholds: $|\Delta E| < 10^{-10} E_h$, $\max |\Delta \mathbf{P}| < 10^{-8}$.

2.4. MP2 and Analysis

The MP2 [14] correlation energy is computed via a four-index integral transformation. Mulliken and Löwdin [15] population analysis and electric dipole moments are also implemented.

2.5. Basis Sets

Built-in data (Table 3) cover H-F; exponents and coefficients are taken directly from PySCF’s basis-set data [16].

Table 3: Built-in basis sets.

Basis	Reference	AO / heavy atom
STO-3G	[17]	5
6-31G*	[18, 19]	15
cc-pVDZ	[20]	15

3. Illustrative Example

The following script computes RHF and MP2 energies of water:

```
1 from molekul.molecule import Molecule; from molekul.atoms import Atom
2 from molekul.basis_sto3g import get_sto3g
3 from molekul.rhf import rhf_scf; from molekul.mp2 import mp2_energy
4 import numpy as np
5
6 B = 1.8897259886 # Ang to Bohr
7 mol = Molecule([Atom('O', [0, 0, .117*B]),
8                 Atom('H', [0, .757*B, -.469*B]), Atom('H', [0, -.757*B, -.469*B])])
9 rhf = rhf_scf(mol, get_sto3g()) # -74.96294667 Eh
10 mp2 = mp2_energy(mol, get_sto3g(), rhf) # -74.99844951 Eh
```

4. Impact

MOLEKUL serves two complementary purposes. As a teaching tool, all algorithms trace to named functions in readable Python, making it suitable for advanced undergraduate and graduate courses in quantum chemistry and computational physics. As an algorithmic-inspection and validation baseline, the 606-test suite and deterministic NumPy [21] implementation allow controlled comparisons of numerical back-ends (e.g. Numba, CuPy) without compiled electronic-structure kernels. Benchmark energies are logged to version-controlled JSON files for full reproducibility.

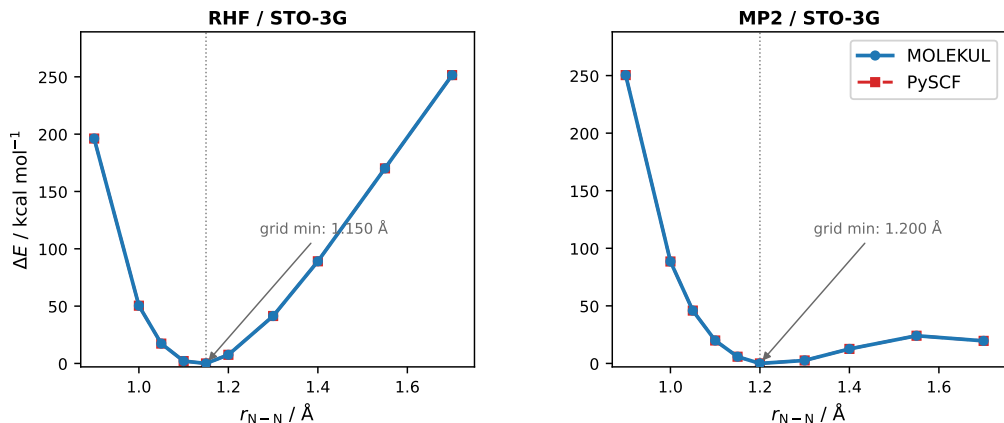


Figure 1: N₂ potential energy scan at RHF/STO-3G (left) and MP2/STO-3G (right). Energies are referenced to the minimum of each curve. MOLEKUL (solid blue) and PySCF (dashed red) are numerically indistinguishable on this scale: $|\Delta E|_{\text{max}} < 2.2 \times 10^{-7} E_h$ across all geometries. The MP2 scan is restricted to $r \leq 1.70 \text{ \AA}$ to avoid the strongly stretched region where single-reference MP2 becomes unreliable. Dotted lines mark the grid-minimum bond length in each panel. Script: `scripts/potential_energy_scan.py`.

5. Conclusions

5.1. Validation Summary

The script `scripts/benchmark_14mol.py` compares MOLEKUL RHF/STO-3G energies against PySCF [1] for 14 closed-shell molecules (H₂ through CH₂O). All 14 pass with maximum deviation $4.9 \times 10^{-8} E_h$ (C₂H₂), consistent with floating-point differences in integral evaluation. Full results are in `outputs/logs/benchmark_14mol.json`.

Figure 1 shows the N₂ potential energy curve computed with MOLEKUL and PySCF at the RHF/STO-3G and MP2/STO-3G levels. Energies are plotted relative to the respective minimum. The two codes are numerically indistinguishable on this scale: the maximum absolute deviation is $2.1 \times 10^{-7} E_h$ (RHF) and $2.2 \times 10^{-7} E_h$ (MP2) across all scan points. Both implementations locate the minimum at the same grid point (1.150 Å at RHF/STO-3G and 1.200 Å at MP2/STO-3G). The MP2 scan is restricted to $r \leq 1.70 \text{ \AA}$ to avoid the strongly stretched region, where single-reference MP2 becomes unreliable.

Table 4 compares MP2 correlation energies against PySCF for eight STO-3G molecules; all agree within $2 \times 10^{-7} E_h$.

Table 4: MP2 correlation energies: MOLEKUL vs. PySCF (STO-3G).

Molecule	$E_{\text{corr}}^{\text{PySCF}} / E_h$	$ \Delta E / E_h$
H ₂	-0.01313807	8×10^{-10}
HF	-0.01734432	3×10^{-9}
H ₂ O	-0.03550283	1×10^{-8}
N ₂	-0.15419856	2×10^{-8}
CO	-0.12852242	1×10^{-7}
NH ₃	-0.04704176	6×10^{-9}
CH ₄	-0.05650741	6×10^{-9}
HCN	-0.12997898	5×10^{-9}

5.2. Known Limitations

Dense N_{AO}^4 ERI storage limits practical use to small molecules ($N_{\text{AO}} \lesssim 50$); no integral screening, density fitting, or integral-direct algorithms are implemented. The SCF driver supports closed-shell RHF only (no UHF or ROHF). Built-in basis data cover H–F; heavier elements are not parametrized. Geometry optimization and harmonic-frequency workflows use finite differences rather than analytic gradients. Excited-state (CIS) and harmonic-frequency modules are experimental pedagogical infrastructure and are not production-validated. No effective core potentials or relativistic corrections are included.

5.3. Summary

MOLEKUL achieves sub-microhartree agreement with PySCF across a 14-molecule benchmark suite and reproduces potential energy curves at the RHF/STO-3G and MP2/STO-3G levels. The code is designed to be extended incrementally—each limitation above represents a well-defined algorithmic addition that students or researchers can implement against the existing test suite.

Software Availability

- Repository: <https://github.com/yazicienis/molekul>
- Archive DOI: <https://doi.org/10.5281/zenodo.19763107>
- License: MIT
- Version archived: v0.1.2

Declaration of Competing Interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT Author Statement

Enis Yazici: Conceptualization, Methodology, Software, Validation, Investigation, Writing – original draft, Writing – review & editing.

Data Availability

The source code, validation scripts, and benchmark logs are available in the public GitHub repository and archived on Zenodo at the DOI given in the Code Metadata table.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

Generative AI tools, including Claude (Anthropic) and ChatGPT (OpenAI), were used for code drafting, refactoring, documentation support, test-generation suggestions, and editorial review. All scientific algorithms, numerical reference values, validation criteria, and reported results were reviewed and verified by the author, who remains responsible for the correctness, originality, and scientific content of the submission.

Acknowledgements

The author thanks the developers of PySCF, whose reference calculations were used throughout validation, and the Basis Set Exchange [16] for tabulated basis-set parameters.

References

- [1] Q. Sun, X. Zhang, S. Banerjee, P. Bao, M. Barbry, N. S. Blunt, N. A. Bogdanov, G. H. Booth, J. Chen, Z.-H. Cui, J. J. Eriksen, Y. Gao, S. Guo, J. Hermann, M. R. Hermes, K. Koh, P. Koval, S. Lehtola, Z. Li, J. Liu, N. Mardirossian, J. D. McClain, M. Motta, B. Mussard, H. Q. Pham, A. Pulkin, W. Purwanto, P. J. Robinson, E. Ronca, E. R. Sayfutyarova, M. Scheurer, H. F. Schurkus, J. E. T. Smith, C. Sun, S.-N. Sun, S. Upadhyay, L. K. Wagner, X. Wang, A. White, J. D. Whitfield, M. J. Williamson, S. Wouters, J. Yang, J. M. Yu, T. Zhu, T. C. Berkelbach, S. Sharma, A. Y. Sokolov, G. K.-L. Chan, Recent developments in the pyscf program package, *The Journal of Chemical Physics* 153 (2) (2020) 024109. doi : 10.1063/5.0006074.
- [2] D. G. A. Smith, L. A. Burns, A. C. Simmonett, R. M. Parrish, M. C. Schieber, R. Galvelis, P. Kraus, H. Kruse, R. Di Remigio, A. Alenaizan, A. M. James, S. Lehtola, J. P. Misiewicz, M. Scheurer, R. A. Shaw, J. B. Schriber, Y. Xie, Z. L. Glick, D. A. Sirianni, J. S. O'Brien, J. M. Waldrop, A. Kumar, E. G. Hohenstein, B. P. Pritchard, B. R. Brooks, I. Schaefer, Henry F., A. Y. Sokolov, K. Patkowski, I. DePrince, A. Eugene, U. Bozkaya, R. A. King, F. A. Evangelista, J. M. Turney, T. D. Crawford, C. D. Sherrill, Psi4 1.4: Open-source software for high-throughput quantum chemistry, *The Journal of Chemical Physics* 152 (18) (2020) 184108. doi : 10.1063/5.0006002.
- [3] F. Neese, F. Wennmohs, U. Becker, C. Riplinger, The orca quantum chemistry program package, *The Journal of Chemical Physics* 152 (22) (2020) 224108. doi : 10.1063/5.0004608.

- [4] R. P. Muller, Pyquante: Quantum chemistry in python, sourceforge.net, accessed: 2026-05-14 (2004).
- [5] A. Szabo, N. S. Ostlund, *Modern Quantum Chemistry: Introduction to Advanced Electronic Structure Theory*, McGraw-Hill, New York, 1989.
- [6] T. Helgaker, P. Jørgensen, J. Olsen, *Molecular Electronic-Structure Theory*, John Wiley & Sons, Ltd, 2000. doi:10.1002/9781119019572.
- [7] L. E. McMurchie, E. R. Davidson, One- and two-electron integrals over cartesian gaussian functions, *Journal of Computational Physics* 26 (2) (1978) 218–231. doi:10.1016/0021-9991(78)90092-X.
- [8] S. F. Boys, Electronic wave functions - i. a general method of calculation for the stationary states of any molecular system, *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 200 (1063) (1950) 542–554. doi:10.1098/rspa.1950.0036.
- [9] C. C. J. Roothaan, New developments in molecular orbital theory, *Rev. Mod. Phys.* 23 (1951) 69–89. doi:10.1103/RevModPhys.23.69.
- [10] G. G. Hall, The molecular orbital theory of chemical valency viii. a method of calculating ionization potentials, *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 205 (1083) (1951) 541–552. doi:10.1098/rspa.1951.0048.
- [11] P. Pulay, Convergence acceleration of iterative sequences. the case of scf iteration, *Chemical Physics Letters* 73 (2) (1980) 393–398. doi:10.1016/0009-2614(80)80396-4.
- [12] P. Pulay, Improved scf convergence acceleration, *Journal of Computational Chemistry* 3 (4) (1982) 556–560. doi:10.1002/jcc.540030413.
- [13] V. R. Saunders, I. H. Hillier, A “level-shifting” method for converging closed shell hartree–fock wave functions, *International Journal of Quantum Chemistry* 7 (4) (1973) 699–705. doi:10.1002/qua.560070407.
- [14] C. Møller, M. S. Plesset, Note on an approximation treatment for many-electron systems, *Phys. Rev.* 46 (1934) 618–622. doi:10.1103/PhysRev.46.618.
- [15] P. Löwdin, On the non-orthogonality problem connected with the use of atomic wave functions in the theory of molecules and crystals, *The Journal of Chemical Physics* 18 (3) (1950) 365–375. doi:10.1063/1.1747632.
- [16] B. P. Pritchard, D. Altarawy, B. Didier, T. D. Gibson, T. L. Windus, New basis set exchange: An open, up-to-date resource for the molecular sciences community, *Journal of Chemical Information and Modeling* 59 (11) (2019) 4814–4820. doi:10.1021/acs.jcim.9b00725.
- [17] W. J. Hehre, R. F. Stewart, J. A. Pople, Self-consistent molecular-orbital methods. i. use of gaussian expansions of slater-type atomic orbitals, *The Journal of Chemical Physics* 51 (6) (1969) 2657–2664. doi:10.1063/1.1672392.
- [18] W. J. Hehre, R. Ditchfield, J. A. Pople, Self-consistent molecular orbital methods. xii. further extensions of gaussian-type basis sets for use in molecular orbital studies of organic molecules, *The Journal of Chemical Physics* 56 (5) (1972) 2257–2261. doi:10.1063/1.1677527.

- [19] P. C. Hariharan, J. A. Pople, The influence of polarization functions on molecular orbital hydrogenation energies, *Theor. Chim. Acta* 28 (1973) 213–222. doi : 10.1007/BF00533485.
- [20] J. Dunning, Thom H., Gaussian basis sets for use in correlated molecular calculations. i. the atoms boron through neon and hydrogen, *The Journal of Chemical Physics* 90 (2) (1989) 1007–1023. doi : 10.1063/1.456153.
- [21] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al., Array programming with NumPy, *Nature* 585 (2020) 357–362. doi : 10.1038/s41586-020-2649-2.